Two-Machine Flow Shop Scheduling to Minimize Total Weighted Completion Time: Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing the total weighted completion time on a two-machine flow shop, denoted $F2 \parallel \sum w_j C_j$, is a classical NP-hard scheduling problem with applications in manufacturing, computing, and logistics. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial permutation S. Scanning S from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block B_k jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from S is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; Total weighted completion time; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Preliminaries

We consider the two-machine flow shop problem $F2 \parallel \sum w_j C_j$. There are n jobs to be processed on two machines M1 and M2 in series: each job j ($j = 1, \ldots, n$) must be processed first on M1 for a_j time units and then on M2 for b_j time units, with no preemption allowed. M1 can process at most one job at a time, and M2 can process at most one job at a time. Each job j has a weight $w_j > 0$. A permutation schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. The completion time of job $\pi(i)$ on M2, denoted $C_{\pi(i)}$, is computed by the standard flow shop recursion:

$$C_{\pi(1)} = a_{\pi(1)} + b_{\pi(1)}, \qquad C_{\pi(i)} = \max\Big\{ \sum_{h=1}^{i} a_{\pi(h)},\ C_{\pi(i-1)} \Big\} + b_{\pi(i)}, \quad i = 2, \ldots, n.$$

The objective is to find a permutation π that minimizes the total weighted completion time $\sum_{j=1}^{n} w_j C_j$. The problem is strongly NP-hard Garey et al. (1976).

In a permutation schedule, the job order on M1 and M2 is identical; for $F2 \parallel \sum w_j C_j$ there always exists an optimal schedule of this form Baker (1974). All n jobs are sequenced by Johnson's rule to obtain a global sequence $S = (j_1, j_2, \ldots, j_n)$ Hall (1998).

Lemma 2.1 *In any optimal schedule, for each $k = 1, \ldots, K$, the jobs belonging to block B_k can only appear within the first k blocks of the final schedule.*

Lemma 2.1 implies that once the k-th block of the final schedule has been processed, all jobs from the first k blocks of S are fully scheduled; consequently, the k-th final block may contain only jobs from $B_k, B_{k+1}, \ldots, B_K$.

Lemma 2.2 *In any optimal schedule, the final job of the k-th final block carries weight $W(B_k)$, for each $k = 1, \ldots, K$. By the block construction rule, $W(B_k)$ is the weight that first appears at the end of B_k in S; thus the condition is automatically satisfied.*

Taken together, the permutation property and these two lemmas determine the structure of any optimal schedule: all jobs are first sequenced globally by Johnson's rule, then partitioned into K blocks by the first occurrence of each new weight; the final schedule respects the block order (block B_k cannot appear beyond the k-th final position); and the last job of each block carries the weight $W(B_k)$.

3 Dynamic Programming Algorithm

The structural properties established in Section 2 enable a dynamic programming algorithm that constructs the schedule by appending jobs block by block. Our approach adapts

the interval-based state representation of Hall (1998), originally devised for the $F2|r_j|C_{\max}$, to the weighted completion time objective.

As described in Section 2, the algorithm takes as input the global Johnson sequence $S = (j_1, \ldots, j_n)$ and its partition into K blocks $B_1, \ldots, B_K$, and processes jobs in block order $B_1, B_2, \ldots, B_K$, appending each job to the end of the current partial permutation. The positional constraint of Lemma 2.1 is automatically satisfied by this discipline: jobs of B_k are appended only after blocks $B_1, \ldots, B_{k-1}$ have been fully processed. The block-end condition is enforced at block boundaries via the check $W(B_k) \leq \ell$ described below. Its correctness follows from the fact that $W(B_k)$ is the weight of the final job of block B_k and $\ell = W(B_{k-1})$; the inequality ensures block weights are non-increasing, a necessary condition for optimality. If violated, no subsequent appending from $B_{k+1}, \ldots, B_K$ can alter the already-fixed weight ordering of the prefix, so the state can be safely discarded. If the check passes, ℓ is updated to $W(B_k)$ for comparison with the next block.

Lemma 3.1 *Within each block B_k, all jobs are sequenced according to the Johnson's rule: for any two jobs $x, y \in B_k$, job x precedes job y if and only if*

$$\min\{a_x, b_y\} \leq \min\{a_y, b_x\},$$

with ties broken by non-decreasing a_j. Since the initial sequence S already satisfies this ordering globally and each B_k is a contiguous subsequence of S, the intra-block Johnson order is automatically preserved by the append-in-block-order discipline.

3.1 State Definition and Dynamic Programming

Each state implicitly corresponds to a partial permutation π uniquely determined by the DP construction path; π is not stored. A state is a 4-tuple $(A, \mathcal{B}, z, \ell)$, where A and $\mathcal{B}$ are the cumulative completion times on M1 and M2, respectively, $z = \mathcal{B}$ is the makespan of the partial schedule, and ℓ is the reference weight $W(B_{k-1})$ of the previously completed block. The initial state is $(0, 0, 0, +\infty)$, corresponding to $\pi = \emptyset$; $\ell = +\infty$ ensures that the first block always passes the weight check.

Let $\mathcal{Y}$ denote the set of states after processing a given prefix of jobs. Given a state $(A, \mathcal{B}, z, \ell) \in \mathcal{Y}$ and a job $j \in B_k$ to append, the transition produces a new state $(A', \mathcal{B}', z', \ell')$ defined by

$$A' = A + a_j, \tag{1}$$

$$\mathcal{B}' = \max(\mathcal{B}, A') + b_j, \tag{2}$$

$$z' = \mathcal{B}', \tag{3}$$

$$\ell' = \begin{cases} W(B_k), & \text{if } j \text{ is the last job of } B_k, \\ \ell, & \text{otherwise.} \end{cases} \tag{4}$$

All four operations take $O(1)$ time. At a block boundary (when the last job of B_k is appended), the constraint $W(B_k) \leq \ell$ is verified; if violated, the candidate state is discarded.

Lemma 3.2 *For any two states $(A, \mathcal{B}, z, \ell)$ and $(A', \mathcal{B}', z', \ell')$ in $\mathcal{Y}$ satisfying $z \leq z'$, the latter state can be removed from $\mathcal{Y}$.*

Proof. Let ρ and ρ' be the partial schedules attaining $(A, \mathcal{B}, z, \ell)$ and $(A', \mathcal{B}', z', \ell')$, respectively. Let σ be an arbitrary suffix schedule for the remaining jobs. Appending σ to ρ yields a complete schedule, and appending σ to ρ' yields another. Since the contribution of σ to the makespan is monotone non-decreasing in the prefix completion times $(A, \mathcal{B})$, the condition $z \leq z'$ guarantees that ρ leads to a makespan no larger than ρ' under any suffix. Therefore ρ dominates ρ'. □

The dominance rule of Lemma 3.2 justifies retaining only states with the minimum z value after processing each job. The complete DP procedure is presented as Algorithm 1.

Algorithm 1 DP_F2_WeightedSum

Input: Jobs J with processing times (a_j, b_j) and weights w_j, $j = 1, \ldots, n$.
Output: The optimal makespan Z^* and its associated optimal permutation π^*.
$S \leftarrow \textsc{JohnsonSort}(J)$
$seen \leftarrow \{w_{S[1]}\}$; $start \leftarrow 1$; $K \leftarrow 0$
for $i = 2$ to n **do**
    if $w_{S[i]} \notin seen$ **then**
        $K \leftarrow K + 1$; $B_K \leftarrow S[start \ldots i]$
        $seen \leftarrow seen \cup \{w_{S[i]}\}$; $start \leftarrow i + 1$
    end if
end for
if $start \leq n$ **then**
    $K \leftarrow K + 1$; $B_K \leftarrow S[start \ldots n]$
end if
$\mathcal{Y} \leftarrow \{(0,\ 0,\ 0,\ +\infty)\}$
for $k = 1$ to K **do**
    $w_k \leftarrow W(B_k)$
    for $i = 1$ to $|B_k|$ **do**
        $j \leftarrow B_k[i]$; $\mathcal{Y}_{\text{new}} \leftarrow \emptyset$
        for each $(A,\ \mathcal{B},\ z,\ \ell) \in \mathcal{Y}$ **do**
            $A' \leftarrow A + a_j$
            $\mathcal{B}' \leftarrow \max(\mathcal{B},\ A') + b_j$
            $z' \leftarrow \mathcal{B}'$

if $i = |B_k|$ **then**
if $w_k > \ell$ **then continue**
end if
$\ell' \leftarrow w_k$
else
$\ell' \leftarrow \ell$
end if
$\mathcal{Y}_{\text{new}} \leftarrow \mathcal{Y}_{\text{new}} \cup \{(A',\, \mathcal{B}',\, z',\, \ell')\}$
end for
$z_{\min} \leftarrow \min\{\, z' \mid (\cdot,\, \cdot,\, z',\, \cdot) \in \mathcal{Y}_{\text{new}} \,\}$
$\mathcal{Y} \leftarrow \{\, s \in \mathcal{Y}_{\text{new}} \mid s.z = z_{\min} \,\}$
end for
end for
$Z^* \leftarrow \min\{\, z \mid (A,\, \mathcal{B},\, z,\, \ell) \in \mathcal{Y} \,\}$
Recover the optimal permutation π^* by backtracking
return Z^*, π^*

3.2 Complexity Analysis

Let $|\mathcal{Y}|_{\max}$ denote the maximum number of states retained after the elimination step in any single iteration. In the append-only regime each parent state produces exactly one child state, so the state count never explodes combinatorially. The per-state transition takes $O(1)$ time (three arithmetic operations and one comparison). With n jobs processed sequentially, the total running time of Algorithm 1 is

$$O\big(|\mathcal{Y}|_{\max} \cdot n\big). \tag{5}$$

The elimination rule of Lemma 3.2 keeps $|\mathcal{Y}|_{\max}$ small in practice: for each distinct $(A, \mathcal{B}, \ell)$ combination, at most one state (with minimum z) is retained. The $W(B_k) \le \ell$ checks at block boundaries further prune infeasible states early. Each state stores four scalar fields, requiring $O(1)$ space per state, and the total space complexity is $O(|\mathcal{Y}|_{\max})$.

The number of blocks is exactly K. Under geometric rounding of weights, $K = O((1/\varepsilon)\log W_{\max})$, so K is bounded by a polynomial in $1/\varepsilon$ independent of n. Moreover, the elimination step can be implemented in $O(|\mathcal{Y}_{\text{new}}|)$ time by a single linear scan that tracks the minimum z'.

Theorem 3.1 *Algorithm 1 finds an optimal schedule for $F2 \parallel \sum w_j C_j$ in $O(|\mathcal{Y}|_{\max} \cdot n)$ time and $O(|\mathcal{Y}|_{\max})$ space.*

Proof. Correctness follows from Lemmas **??**–3.1, the block partitioning rule of Section 2, the state transition (1)–(4), and the dominance rule of Lemma 3.2. Step 1 (Johnson sorting and block partitioning) takes $O(n \log n)$ time. Steps 2 and 4 require $O(1)$ and $O(|\mathcal{Y}|)$ time, respectively. In Step 3, the outer loop over k and inner loop over i together visit each of the n jobs exactly once. For each job, the algorithm iterates over $|\mathcal{Y}|$ states and performs $O(1)$ work per state, then applies elimination in $O(|\mathcal{Y}_{\text{new}}|)$ time. Hence the total cost is $O(|\mathcal{Y}|_{\max} \cdot n)$. The space bound follows because only $\mathcal{Y}$ and $\mathcal{Y}_{\text{new}}$ are stored, each of size at most $|\mathcal{Y}|_{\max}$. □

4 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling.* Wiley, New York.

Garey, M. R., Johnson, D. S., and Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, 1(2):117–129.

Hall, L. A. (1998). Approximability of flow shop scheduling. *Mathematical Programming*, 82(1–2):175–190.